Network System Design
CS6100
Tutorial - 07

Student Details
Name: R Abinav
Roll No: ME23B1004
Date: March 06, 2026

1 Source Code

Listing 1: ip packet fragmentation in rust

```rust
fn main() {
    let mtu1: usize = 4020; // network 1 mtu (large)
    let mtu2: usize = 1500; // network 2 mtu (small, path mtu)
    let ip_header: usize = 20;
    let original_payload: usize = 4000;
    println!("ip packet fragmentation demo");
    println!("network 1 mtu: {} bytes", mtu1);
    println!("network 2 mtu: {} bytes (bottleneck)", mtu2);
    println!("ip header size: {} bytes", ip_header);
    println!("original payload: {} bytes", original_payload);
    println!("total datagram: {} bytes\n", original_payload +
        ip_header);
    fragment_packet(original_payload, mtu2, ip_header);
}

struct Fragment {
    fragment_number: usize,
    data_offset: usize,  // byte offset in original payload
    data_length: usize,  // bytes of payload in this fragment
    total_length: usize, // data + header
    offset_field: usize, // ip offset field (data_offset / 8)
    mf_flag: u8,         // more fragments flag
}

fn fragment_packet(payload_size: usize, path_mtu: usize,
    ip_header: usize) {
    let max_data_per_fragment = path_mtu - ip_header;
    // align max data to 8-byte boundary (required by ip
        fragmentation spec)
    let max_data_aligned = (max_data_per_fragment / 8) * 8;
```

```rust
    println!("max data per fragment (aligned to 8 bytes): {} bytes\n",
        max_data_aligned);
    let mut fragments: Vec<Fragment> = Vec::new();
    let mut bytes_sent: usize = 0;
    let mut fragment_number: usize = 1;
    while bytes_sent < payload_size {
        let remaining = payload_size - bytes_sent;
        // pick full chunk or whatever remains
        let data_length = if remaining > max_data_aligned {
            max_data_aligned
        } else {
            remaining
        };
        let is_last = bytes_sent + data_length >= payload_size;
        let mf_flag: u8 = if is_last { 0 } else { 1 };
        // offset field = bytes already sent / 8
        let offset_field = bytes_sent / 8;
        fragments.push(Fragment {
            fragment_number,
            data_offset: bytes_sent,
            data_length,
            total_length: data_length + ip_header,
            offset_field,
            mf_flag,
        });
        bytes_sent += data_length;
        fragment_number += 1;
    }
    // print table header
    println!(
        "{:<12} {:<16} {:<14} {:<14} {:<10}",
        "fragment #", "total length", "data length", "offset field", "mf flag"
    );
    println!("{}", "-".repeat(66));
    for f in &fragments {
        println!(
            "{:<12} {:<16} {:<14} {:<14} {:<10}",
            f.fragment_number, f.total_length,
            f.data_length, f.offset_field, f.mf_flag
        );
    }
    println!();
    // print detailed breakdown per fragment
    for f in &fragments {
        let byte_start = f.data_offset;
        let byte_end = f.data_offset + f.data_length - 1;
        println!("fragment {}", f.fragment_number);
        println!("  data sent    : {} bytes (bytes {} to {})",
            f.data_length, byte_start, byte_end);
```

```rust
        println!("  total size    : {} bytes", f.total_length);
        println!("  mf flag       : {} ({})", f.mf_flag,
            if f.mf_flag == 1 { "more fragments coming" } else { "last fragment" });
        println!("  offset field : {} (calculation: {} / 8 = {})",
            f.offset_field, f.data_offset, f.offset_field);
        println!();
    }
    // summary
    let total_data: usize = fragments.iter().map(|f| f.data_length).sum();
    let total_bytes: usize = fragments.iter().map(|f| f.total_length).sum();
    println!("summary");
    println!("  total fragments created  : {}", fragments.len());
    println!("  total payload reassembled: {} bytes", total_data);
    println!("  total bytes transmitted  : {} bytes (includes {} headers)",
        total_bytes, fragments.len() * ip_header);
    println!("  overhead from headers    : {} bytes",
        fragments.len() * ip_header);
}
```

2 Output

```
abinav@Abinavs-MacBook-Air-97 tutorial-07 % cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
     Running `target/debug/tutorial-07`
ip packet fragmentation demo
network 1 mtu: 4020 bytes
network 2 mtu: 1500 bytes (bottleneck)
ip header size: 20 bytes
original payload: 4000 bytes
total datagram: 4020 bytes

max data per fragment (aligned to 8 bytes): 1480 bytes

fragment #   total length     data length    offset field   mf flag
-----------------------------------------------------------------
1            1500             1480           0              1
2            1500             1480           185            1
3            1060             1040           370            0

fragment 1
  data sent    : 1480 bytes (bytes 0 to 1479)
  total size   : 1500 bytes
  mf flag      : 1 (more fragments coming)
  offset field : 0 (calculation: 0 / 8 = 0)

fragment 2
  data sent    : 1480 bytes (bytes 1480 to 2959)
  total size   : 1500 bytes
  mf flag      : 1 (more fragments coming)
  offset field : 185 (calculation: 1480 / 8 = 185)

fragment 3
  data sent    : 1040 bytes (bytes 2960 to 3999)
  total size   : 1060 bytes
  mf flag      : 0 (last fragment)
  offset field : 370 (calculation: 2960 / 8 = 370)

summary
  total fragments created  : 3
  total payload reassembled: 4000 bytes
  total bytes transmitted  : 4060 bytes (includes 60 headers)
  overhead from headers    : 60 bytes
abinav@Abinavs-MacBook-Air-97 tutorial-07 %
```

Figure 1: output of ip packet fragmentation program showing fragment table and per-fragment breakdown